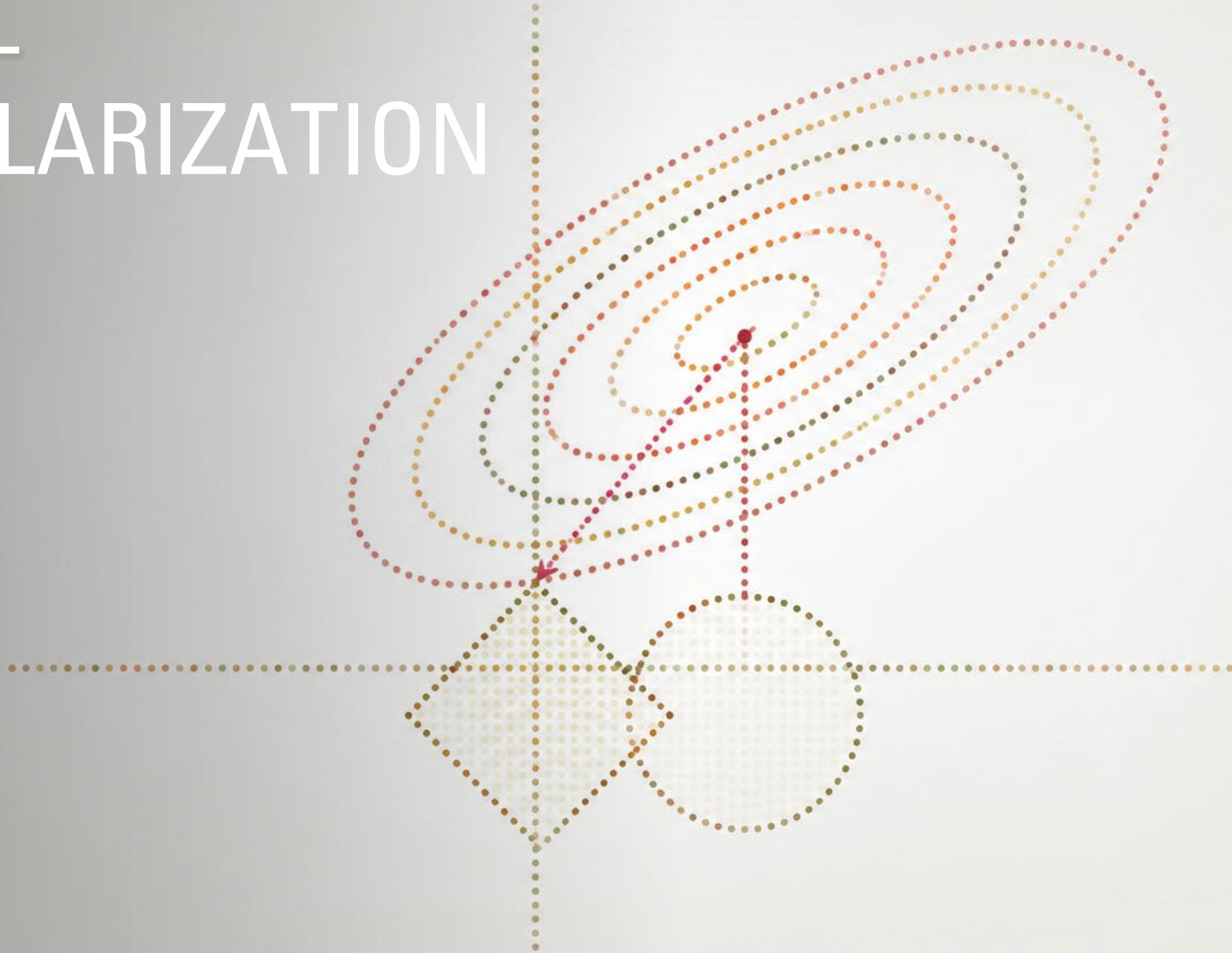

REGULARIZATION

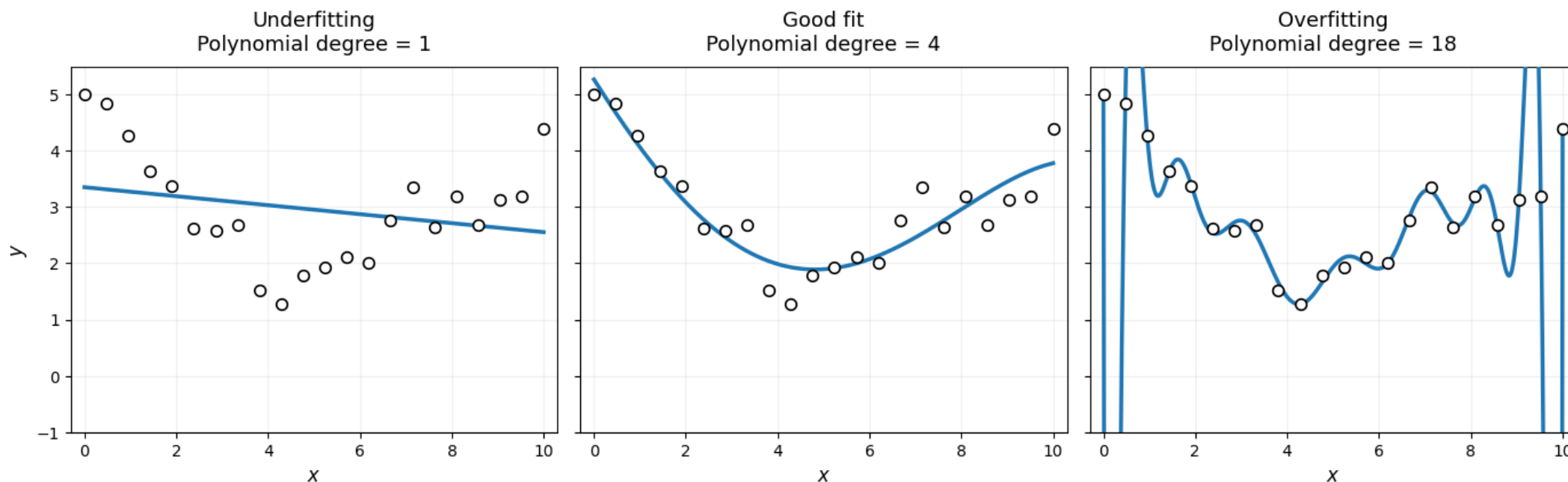




WHY DO WE NEED REGULARIZATION?

- Recall: Fit

Polynomial Regression: Underfitting vs. Good Fit vs. Overfitting



REGULARIZATION

- **Regularization:** techniques that constrain a model to reduce overfitting and improve generalization
- Two broad families:
 - **Explicit** – add a penalty term directly to the loss function
 - **Implicit** – constrain the model through the algorithm or training process

EXPLICIT REGULARIZATION

- Intentionally and directly add a regularization term to the loss function
- Explicitly modify the optimization objective
- Examples (not exhaustive, but these are the most common):
 - L1 Regularization (Lasso)
 - L2 Regularization (Ridge)
 - Elastic Net (combine L1 and L2 regularization)

IMPLICIT REGULARIZATION

- Any regularization method that does not explicitly modify the cost or objective function
 - Can be a byproduct of an algorithm, the learning process, or other choices made in training
 - Examples (not exhaustive):
 - Early stopping (in gradient-based optimization, stop training before convergence is reached)
 - Dropout (randomly ignoring some parts of a neural network)
 - Pruning (simplifying a grown decision tree)
 - Data augmentation (adding synthetic data to increase the sample size)
-

Example of Explicit Regularization in Linear Regression

LEAST SQUARES: THE BASELINE

- Least squares cost function: finds the β 's that minimize

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

L1 PENALTY: LASSO

- Least squares cost function

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

Tuning parameter that controls relative impact of two criteria

- Lasso** linear regression cost function

$$J_{lasso}(\beta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j|, \quad (\alpha > 0)$$

Tries to fit the data well
(find β s that move the prediction close to the observed data)

Constrains the model
(pushes β s to be close to or exactly equal to 0)

What happens if $\alpha = 0$ or $\alpha \approx \infty$?

ℓ_1 VS ℓ_2 PENALTIES

ℓ_1 penalty • Lasso linear regression cost function

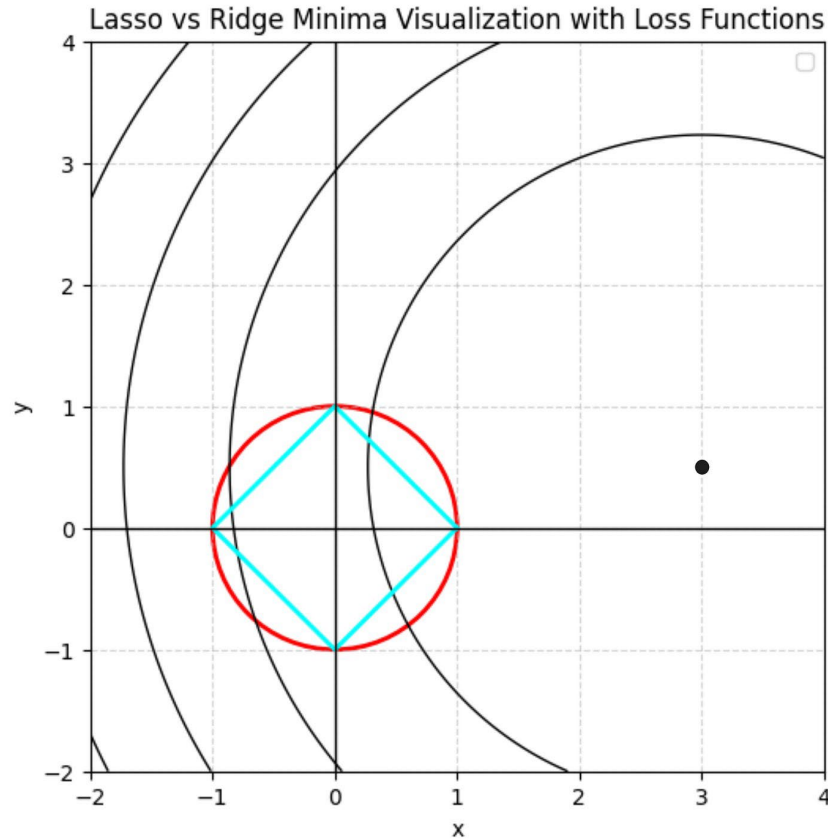
$$J_{lasso}(\beta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j|, \quad (\text{where } \alpha > 0)$$

ℓ_2 penalty • Ridge linear regression cost function

$$J_{ridge}(\beta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \alpha \sum_{j=1}^p (\beta_j)^2, \quad (\text{where } \alpha > 0)$$

Constrains the model
(pushes β s to be close to but NOT
exactly equal to 0)

LASSO V. RIDGE VISUALIZATION



Let's explore this further in
Desmos:
<https://www.desmos.com/3d>

<https://www.vizuaranewsletter.com/p/if-regularization-is-easy-can-you>: This
is a great source for an intuitive description!

RIDGE VS LASSO

	Lasso (ℓ_1)	Ridge (ℓ_2)
Coefficients	Can be driven to exactly 0	Shrink toward 0, never exactly 0
Variable selection	Built in	None — keeps every feature
Correlated features	Picks one somewhat arbitrarily; estimates unstable	Spreads weight across them; estimates stable
Performs best when	Many features contribute little to prediction	Many features contribute, and are correlated

Neither method universally dominates — the right choice depends on how many features actually matter.

CHOOSING THE TUNING PARAMETER

- α controls the bias–variance tradeoff
 - $\alpha \rightarrow 0$: back to ordinary least squares (low bias, high variance)
 - $\alpha \rightarrow \infty$: all coefficients shrink toward 0 (high bias, low variance)
 - Choose α by k-fold cross-validation: fit over a grid of α values, pick the one minimizing CV error
 - The regularization path plots coefficients against α — useful for seeing which features survive
 - Always standardize predictors first: penalties are not scale-invariant, so units change the answer
 - The intercept β_0 is not penalized
-

ELASTIC-NET REGULARIZATION

- Combination of Ridge and Lasso
 - The idea is to capitalize on the strengths of both methods

$$J_{en}(\beta) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \alpha_1 \sum_{j=1}^p |\beta_j| + \alpha_2 \sum_{j=1}^p (\beta_j)^2$$

- If $\alpha_1 = 0$ (and $\alpha_2 \neq 0$), elastic net becomes Ridge
 - If $\alpha_2 = 0$ (and $\alpha_1 \neq 0$), elastic net becomes Lasso
 - If both are 0, we're back to vanilla least squares
-

IN PRACTICE: FITTING REGULARIZED MODELS

- Python — scikit-learn
 - `make_pipeline(StandardScaler(), LassoCV(cv=10)).fit(X, y)`
 - R — glmnet
 - `cv.glmnet(X, y, alpha = 1)`
 - alpha: 1 = Lasso, 0 = Ridge, between = Elastic Net
 - Penalty-weight conventions differ between packages
-

REGULARIZATION GENERALLY

- Regularization is a general idea, not a linear-regression trick
- Used in logistic regression, neural networks, decision trees, and more
- Implicit forms need no penalty term: early stopping, dropout, pruning
- Applies to weights, complexity, or the training process itself

REGULARIZED MODELS IN SCIKIT-LEARN

- By default: LogisticRegression, SVC, MLP
- By design: Ridge, Lasso, ElasticNet
- By complexity: tree pruning, boosting, early stopping
- No penalty: LinearRegression

KEY TAKEAWAYS

- Regularization is the dial that controls model complexity
 - Lasso: sparse coefficients, built-in variable selection
 - Ridge: stable shrinkage, handles correlated features
 - Elastic Net: the middle ground
 - Standardize first, then tune α by cross-validation
-